

感谢各位同学参与课程评教

(浏览器链接)

<https://evaluation.shanghaitech.edu.cn>

期末课程评教入口二维码



EE115B *Digital Circuits*

Instructor: Chenxi Xiao

Part of slides are from

© Pearson Education

Hengzhao Yang

Zhifeng Zhu

Yajun Ha



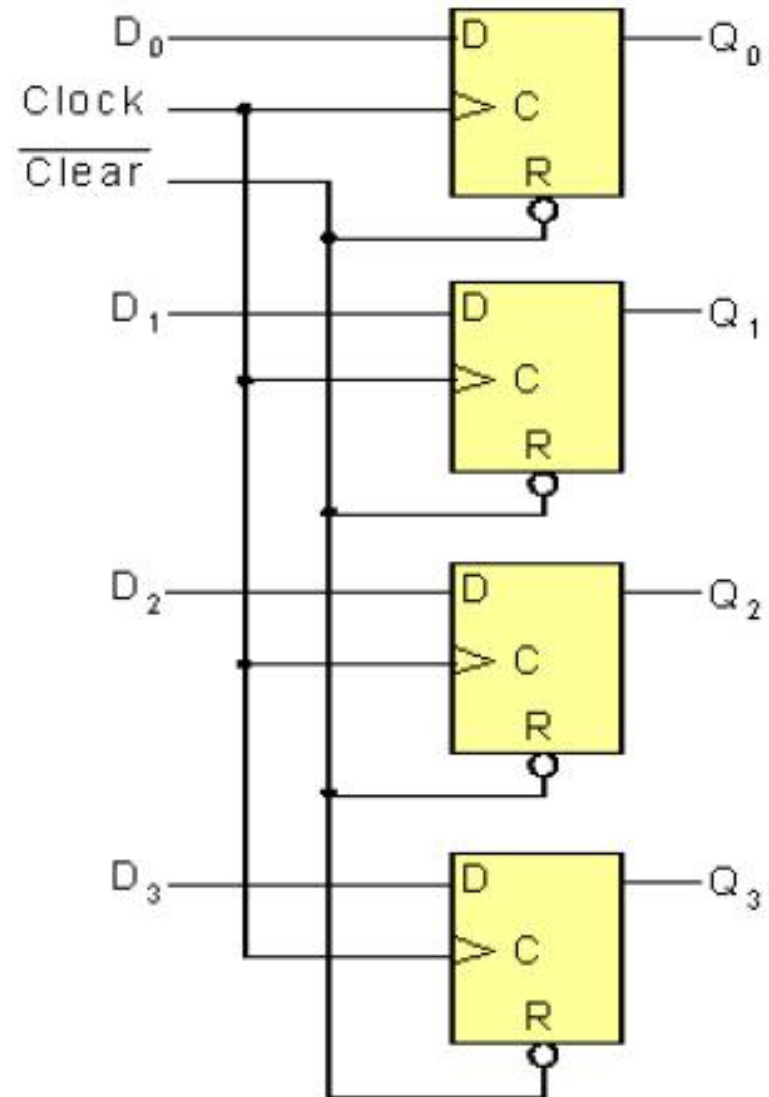
上海科技大学
ShanghaiTech University

Shift Register

Counter

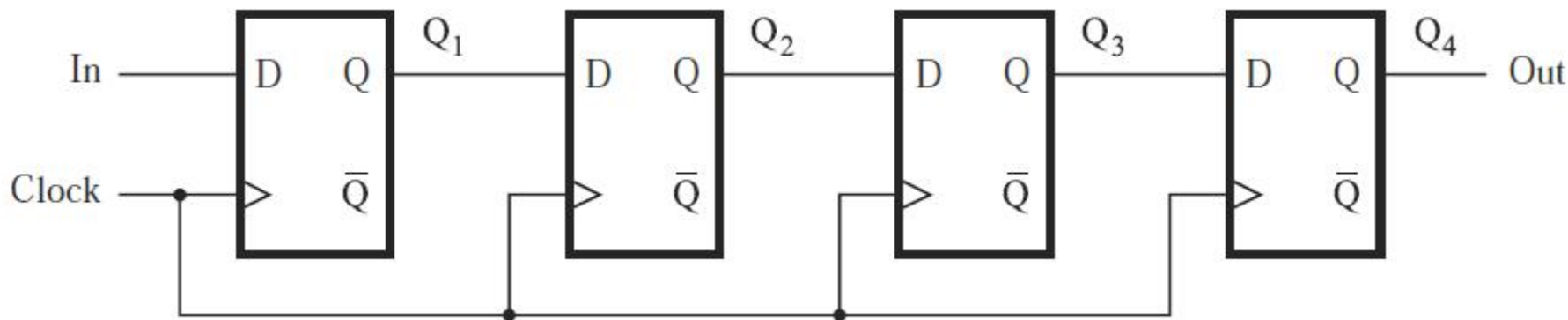
Registers

- An N-bit register is a set of N flip-flops used to store N bits of information.
- Example: 4-bit register
 - One D flip-flop for each bit
 - Same Clock and Clear signals for all 4 D flip-flops
 - Input: D_0 - D_3
 - Output: Q_0 - Q_3



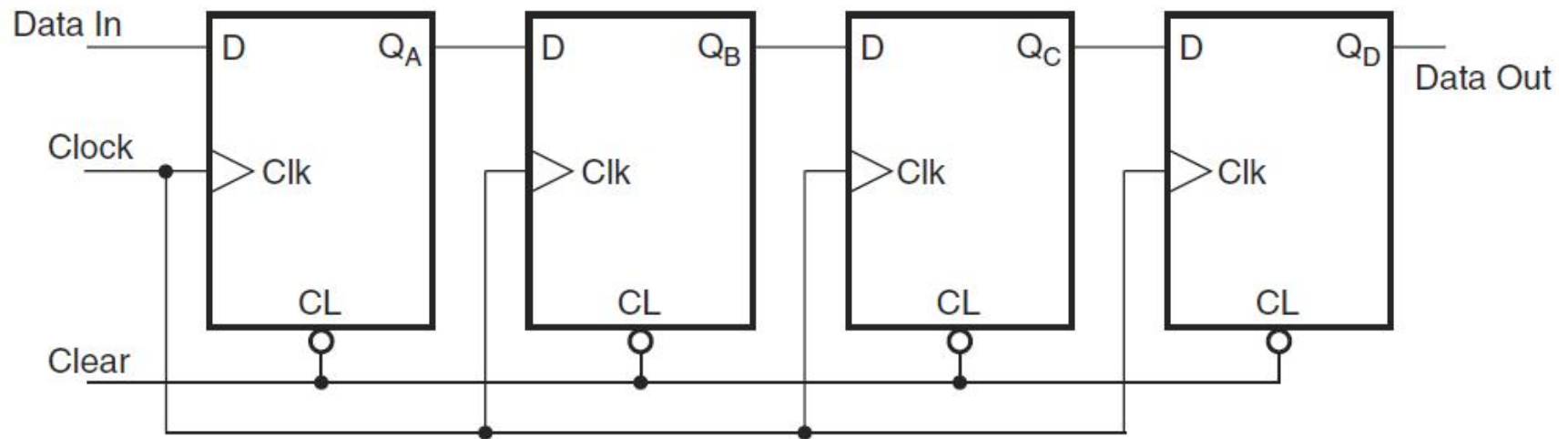
Shift registers

- Shift register: a register with the ability to shift its contents
- Example: 4-bit shift register



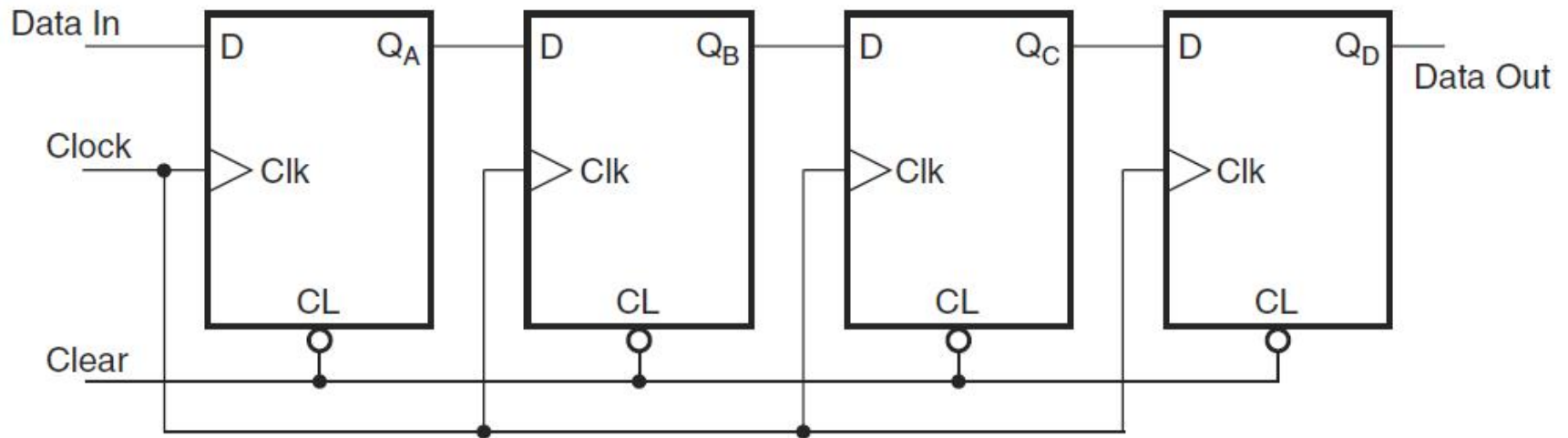
Example: 4-bit shift register

- Circuit diagram (with asynchronous clear)

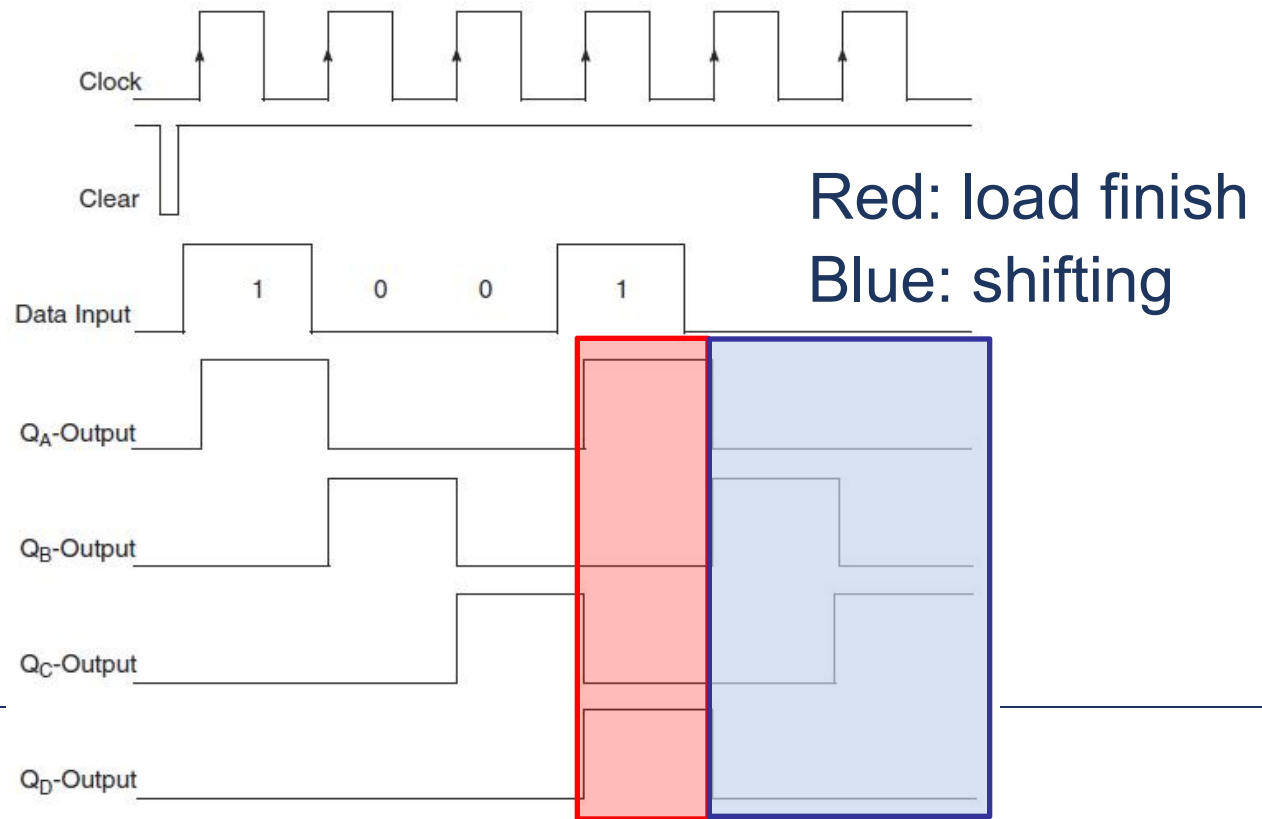
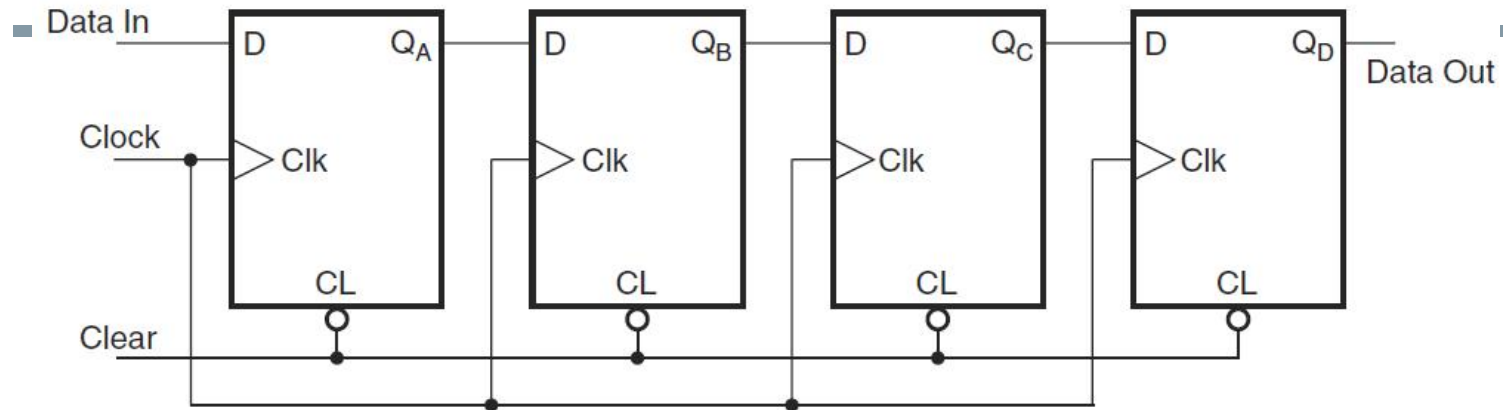


Example: 4-bit shift register

Exercise: draw its timing diagram?
Input is 1001, starting with a clear.
Draw QA QB QC QD.

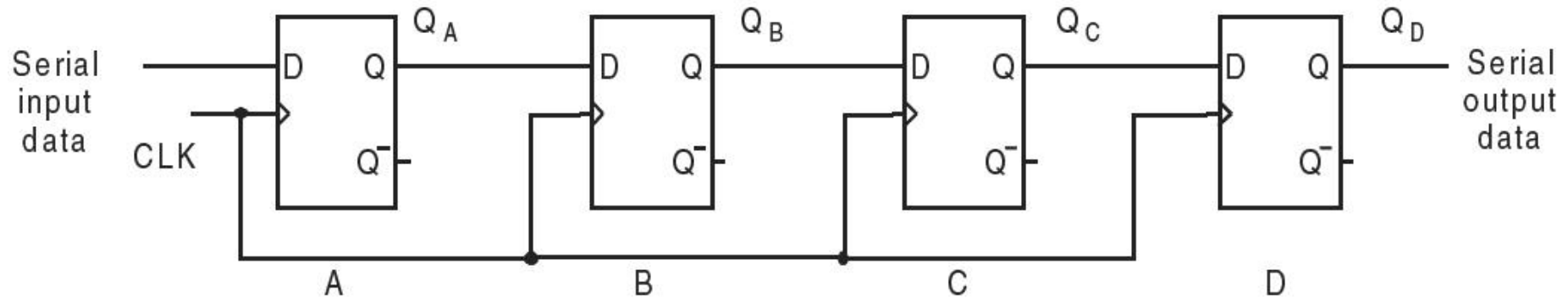


Example: 4-bit shift register

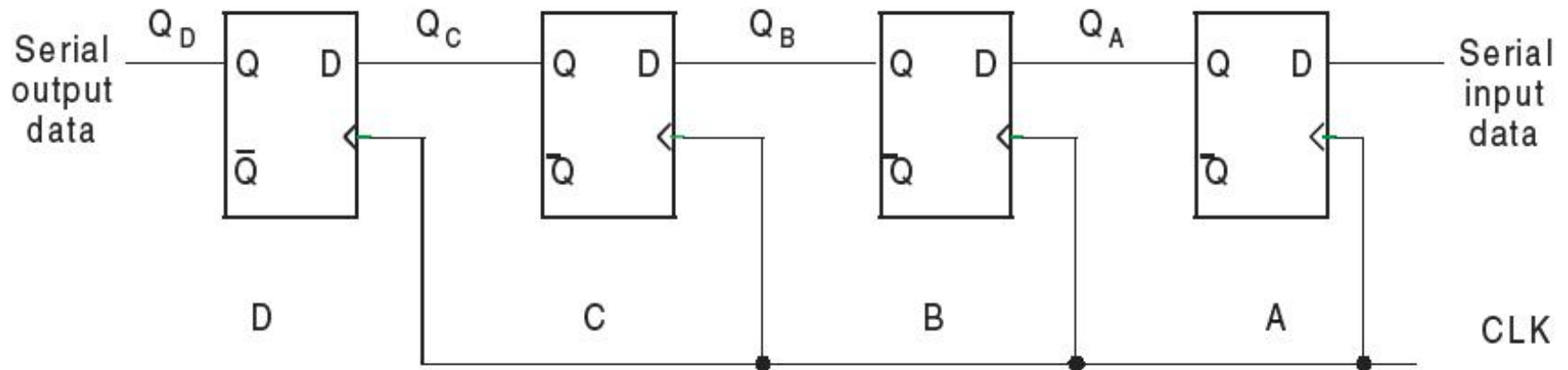


4-bit registers: shift-right and shift-left

Shift-right: need to shift in from low bits.



Shift-left: need to shift in from high bits



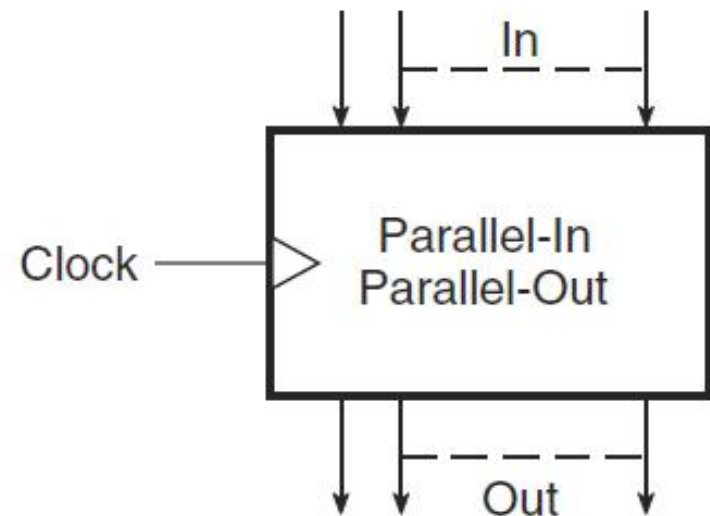
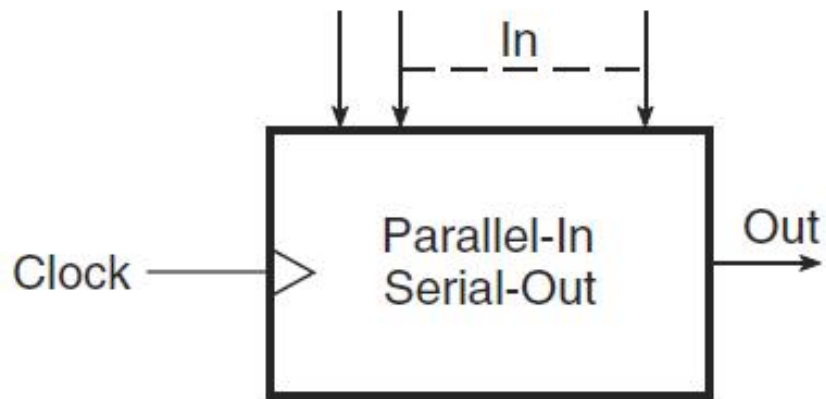
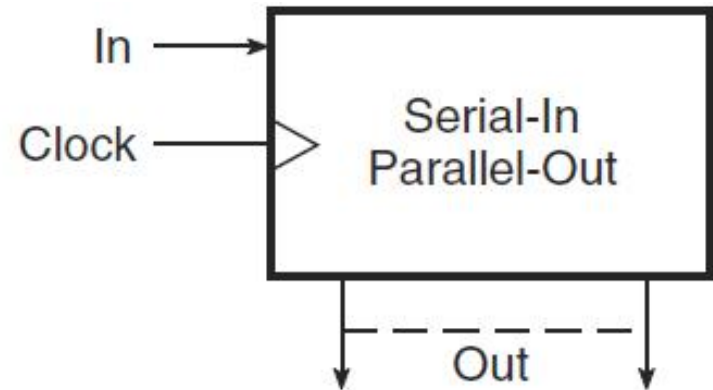
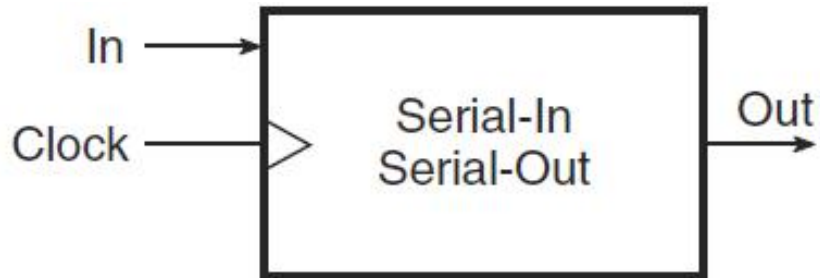
Shift register types

4 types depending on the method to load and read data

- Serial-in serial-out (SISO)
- Serial-in parallel-out (SIPO)
- Parallel-in serial-out (PISO)
- Parallel-in parallel-out (PIPO)

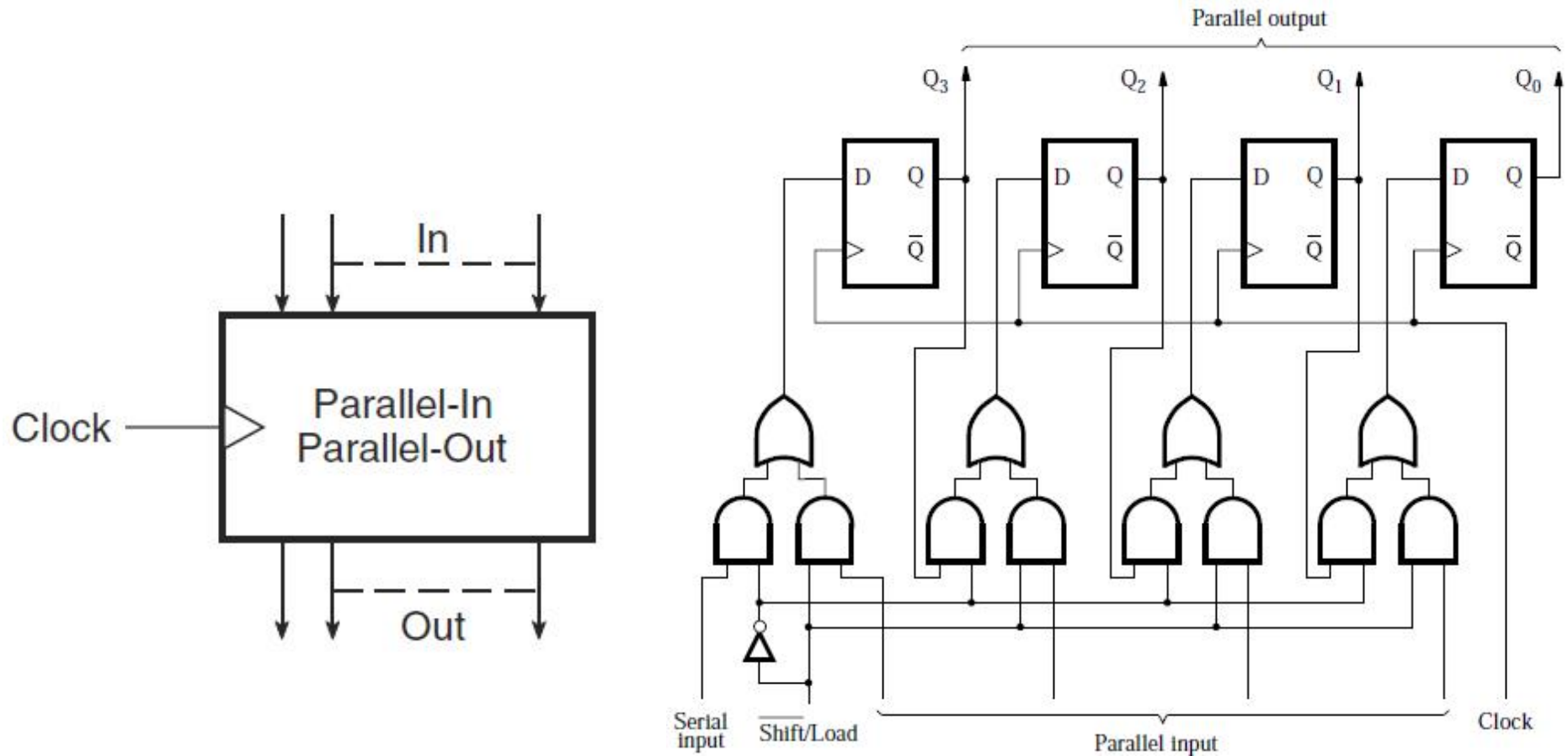


Shift register types



Example: parallel input, parallel output

Example: parallel input, parallel output



Counters

Asynchronous (serial or ripple): different stages are driven by different clocks

- Up-counters
- Down-counters
- Usually only the first FF is connected to CLK.
- Simple design, propagation delay

Synchronous (parallel): different stages are driven by the same clock

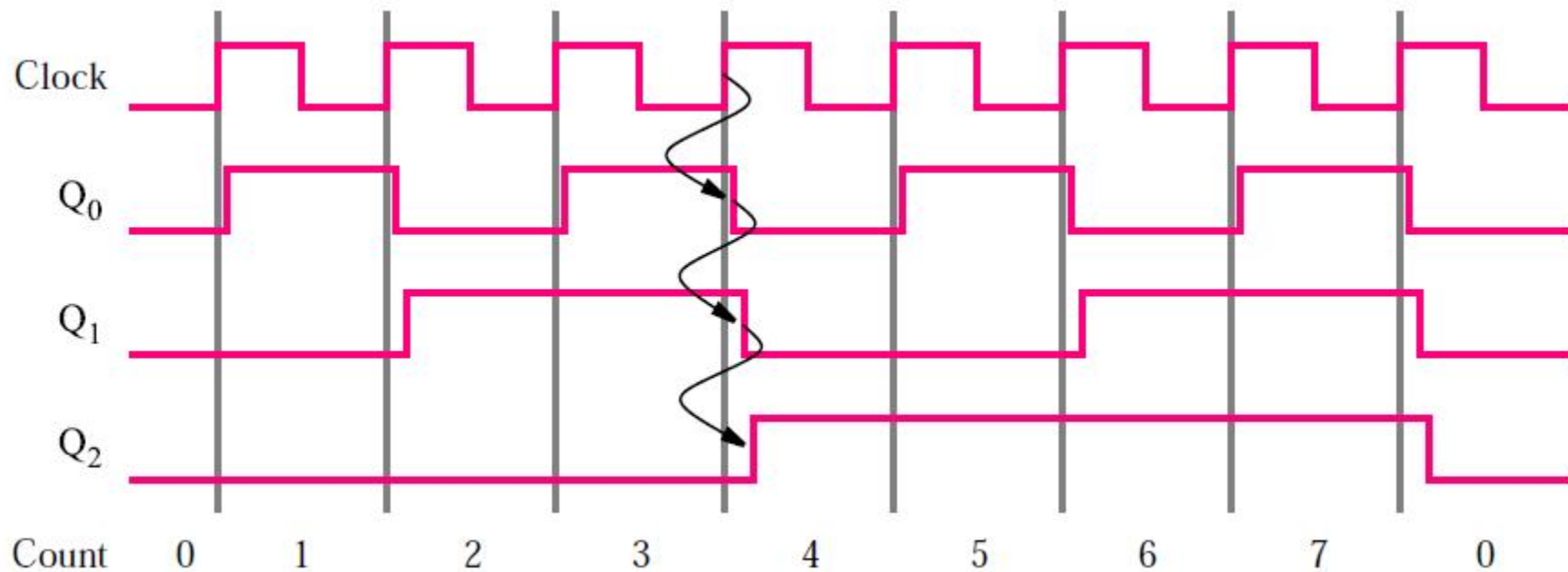
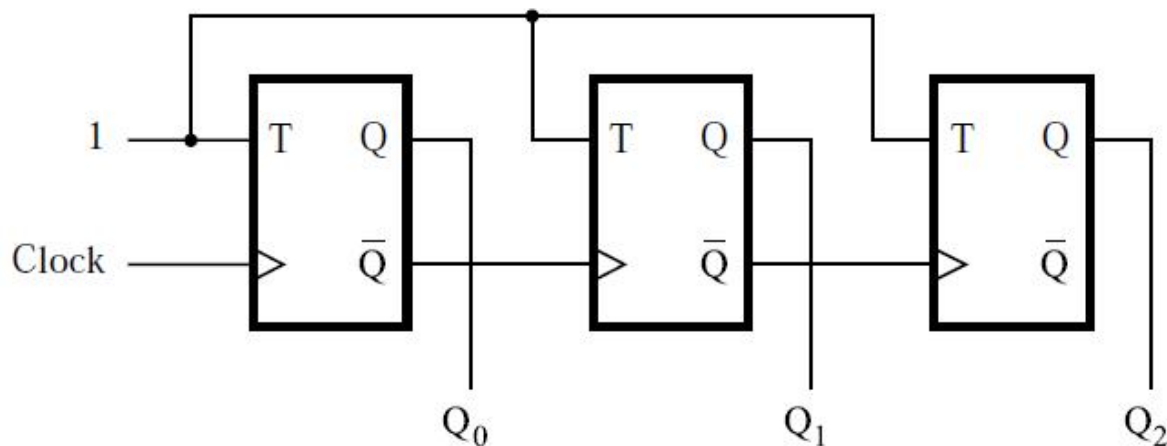
- Up-counters
- Down-counters
- Complex design, faster speed.



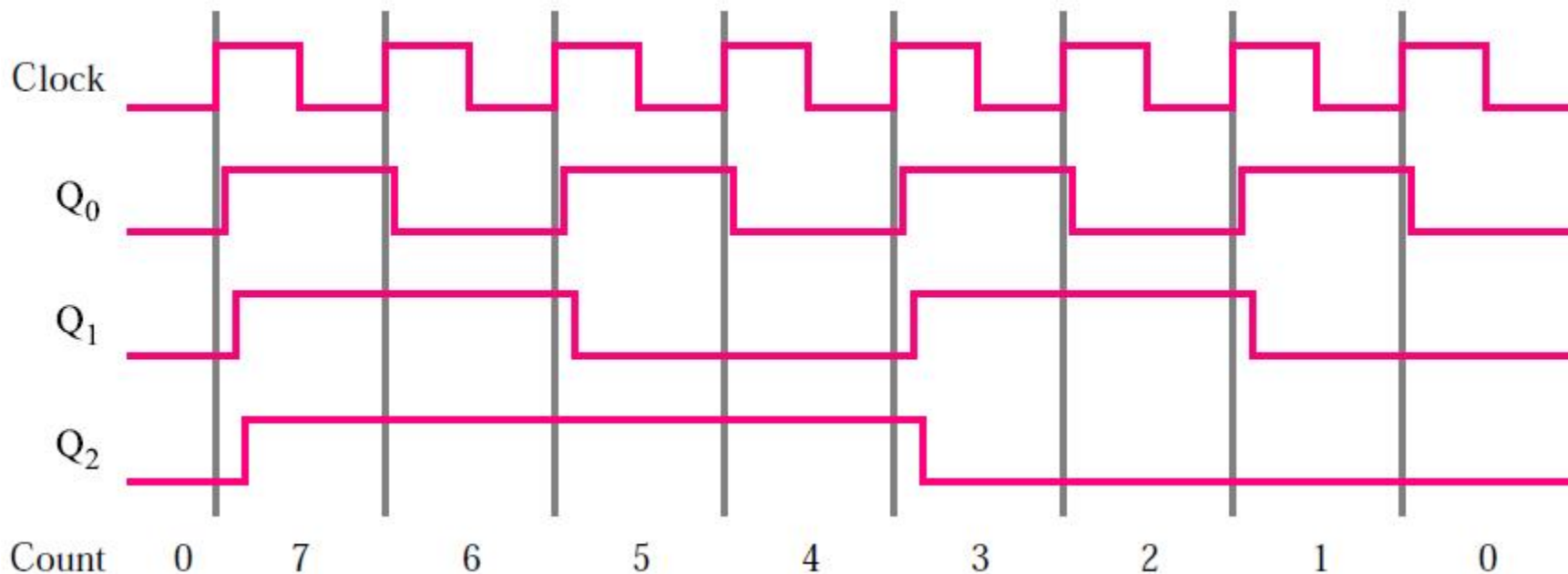
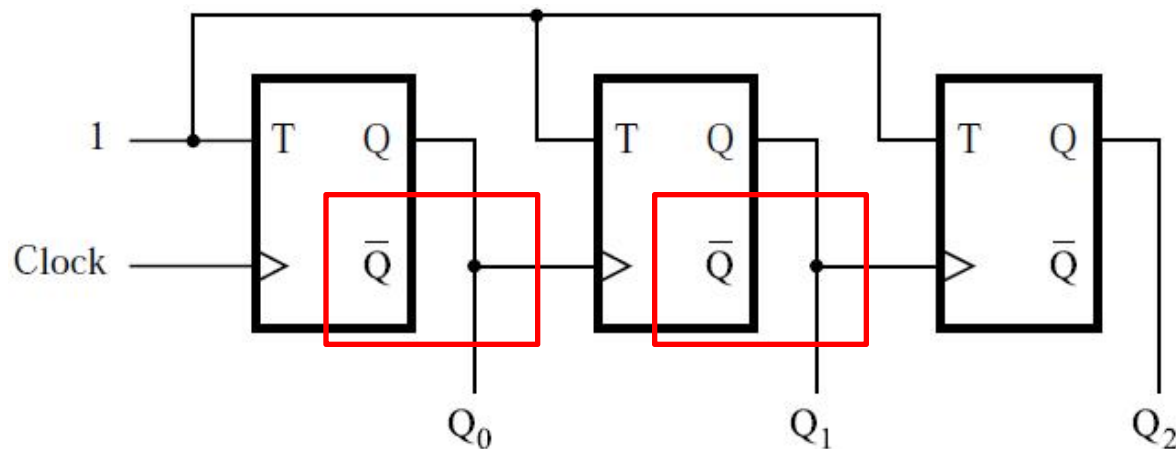
- **Modulus** of a counter is the number of unique states through which the counter will sequence.
- Counters can be designed with modulus less than 2^n , where n is the number of FF.



Asynchronous up-counter with T flip-flops



Asynchronous down-counter with T flip-flops

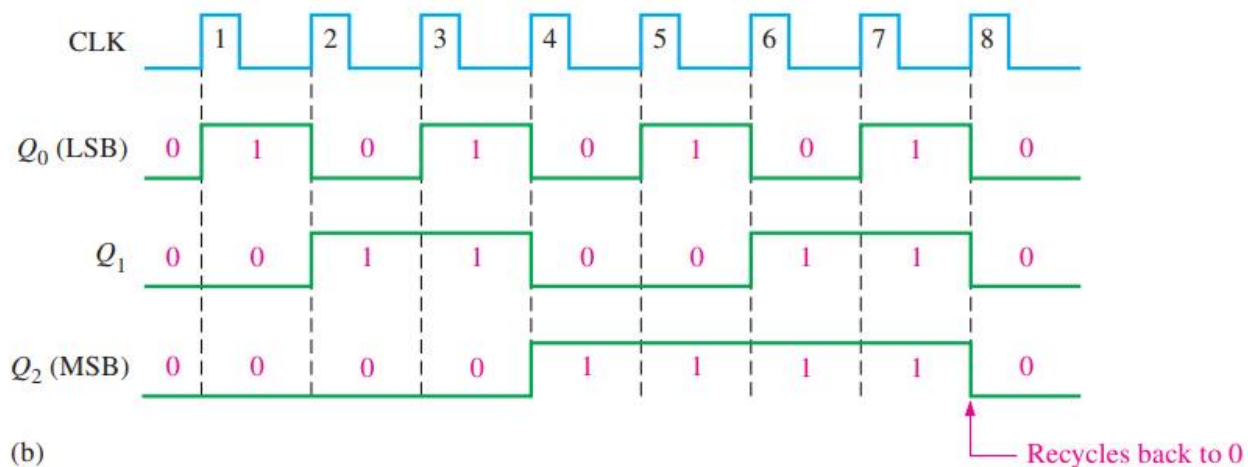
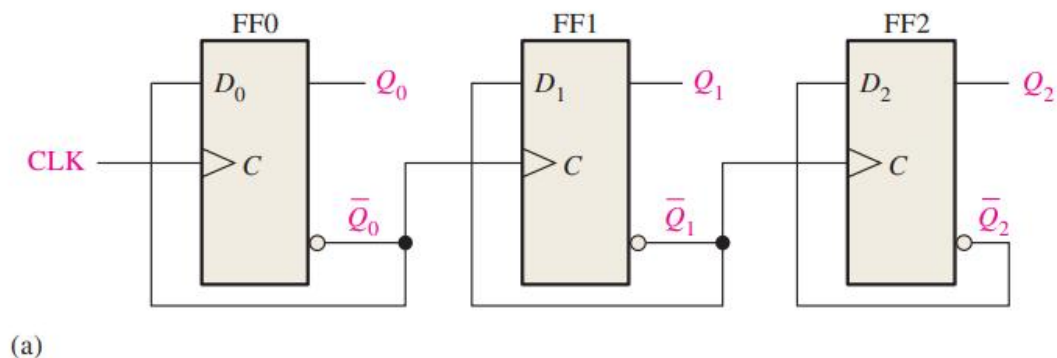


Asynchronous up-counter with D flip-flops

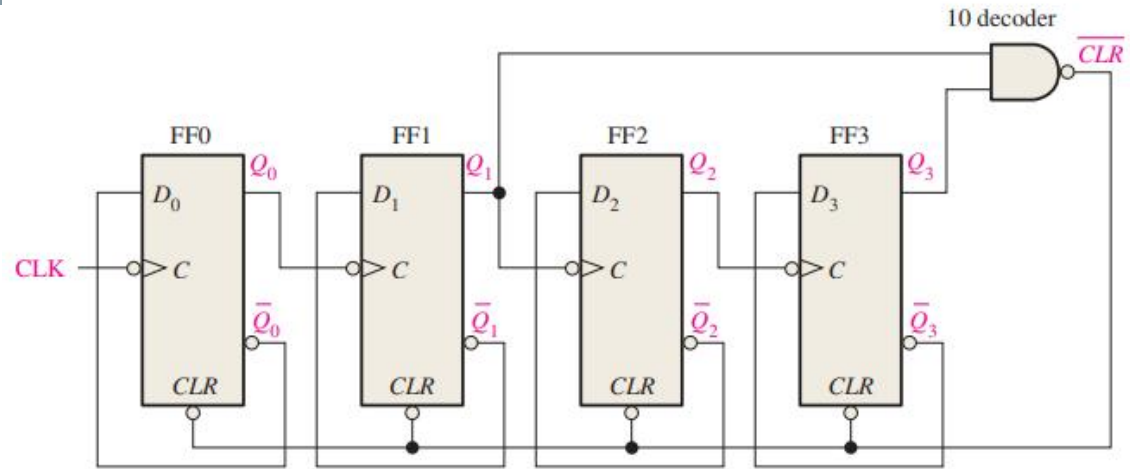
TABLE 9-2

State sequence for a 3-bit binary counter.

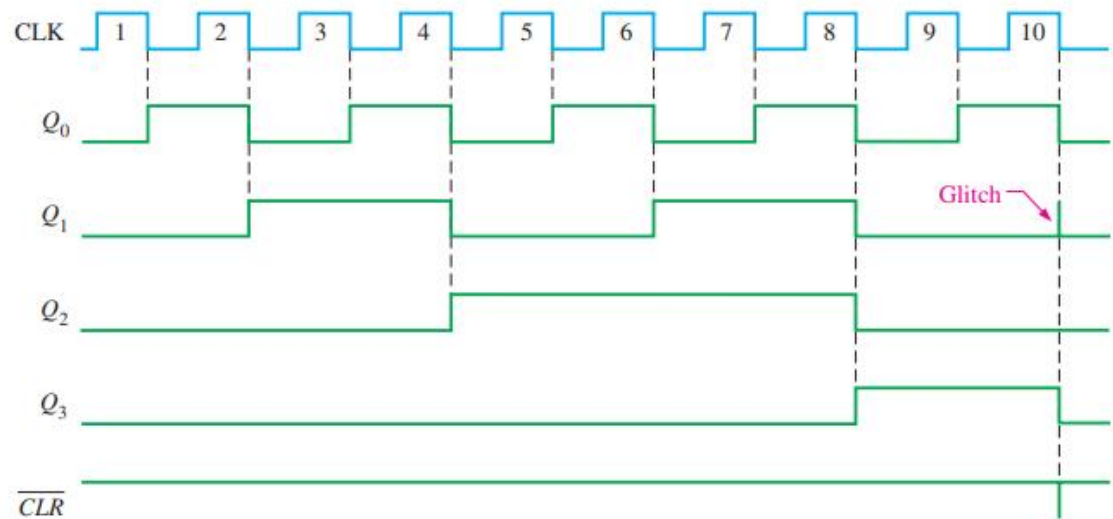
Clock Pulse	Q_2	Q_1	Q_0
Initially	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8 (recycles)	0	0	0



- Truncate unnecessary states.
- When 10 is detected, generate a reset.
- May generate a pulse that lasts very short (Glitch).



(a)



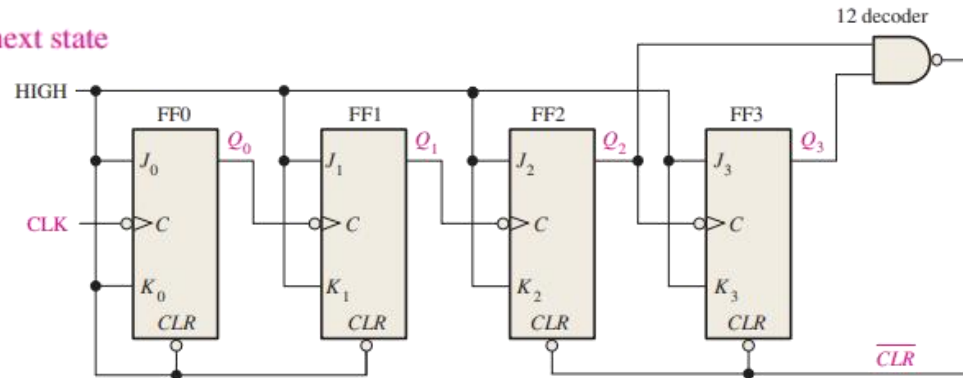
(b)

Asynchronous Modulus 12 Counter

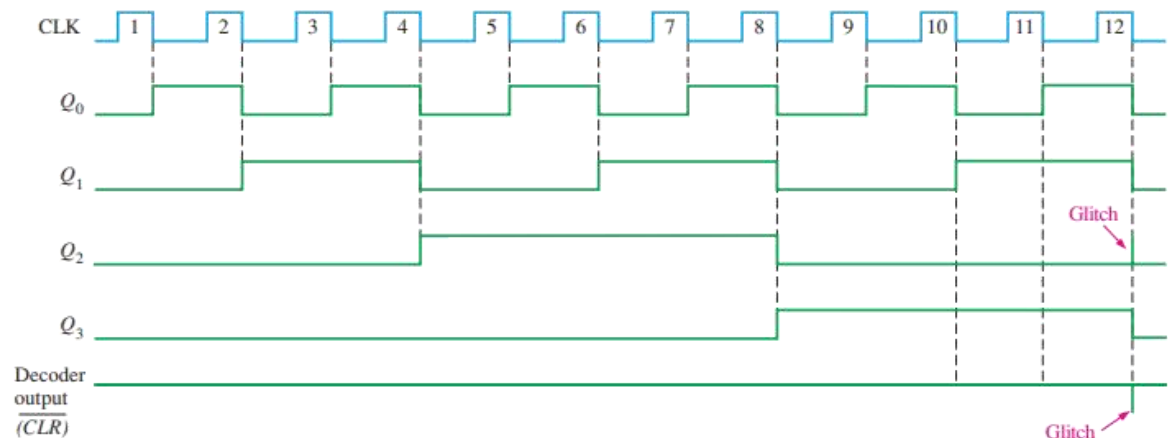
Q_3	Q_2	Q_1	Q_0
0	0	0	0
.	.	.	.
.	.	.	.
.	.	.	.
1	0	1	1
1	1	0	0

Recycles

Normal next state



(a)



(b)



Synchronous 3-bit up-counter with JK flip-flops

A **synchronous counter** is one in which all the flip-flops in the counter are clocked at the same time by a common clock pulse

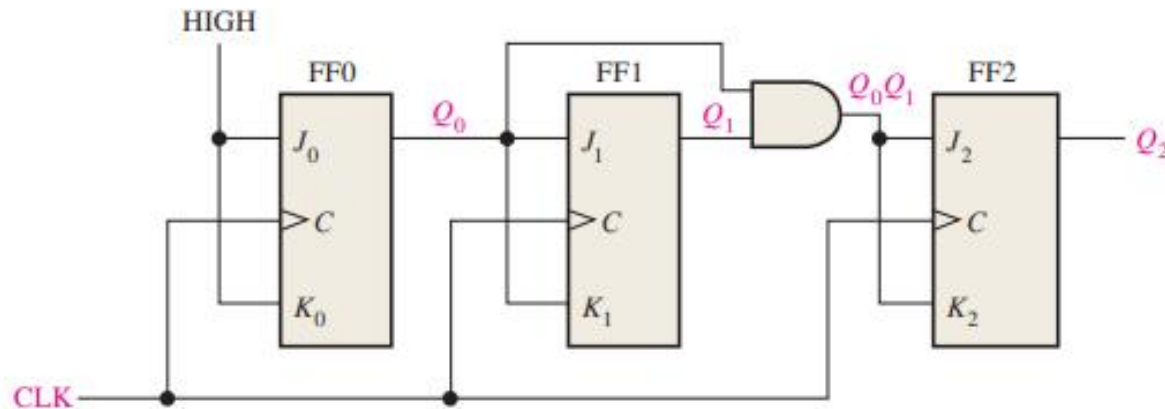


FIGURE 9-15 A 3-bit synchronous binary counter. Open file F09-15 to verify the operation.

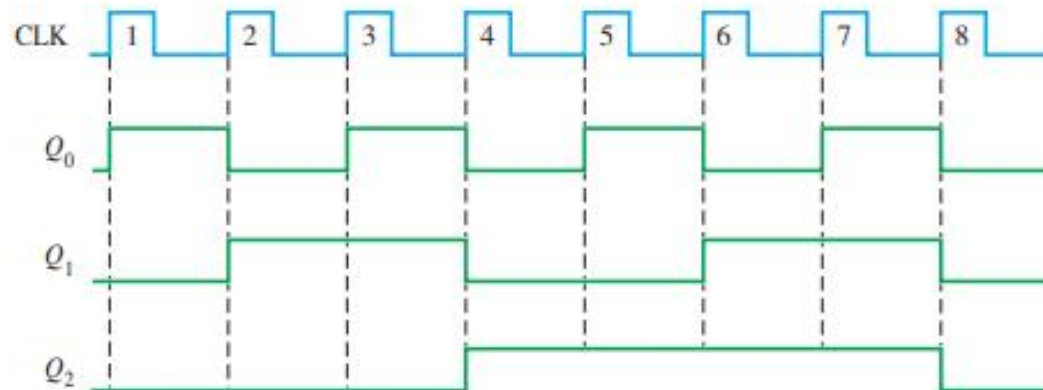


FIGURE 9-16 Timing diagram for the counter of Figure 9-15.

Synchronous up-counter with T flip-flops

Bit pattern: a given stage changes only when all the preceding stages satisfy $Q=1$

- Q_0 changes at each clock positive edge
- Q_1 changes only when $Q_0=1$
- Q_2 changes only when $Q_0=1$ AND $Q_1=1$

T input

$$T_0 = 1$$

$$T_1 = Q_0$$

$$T_2 = Q_0 Q_1$$

$$T_3 = Q_0 Q_1 Q_2$$

.

.

.

$$T_n = Q_0 Q_1 \cdots Q_{n-1}$$

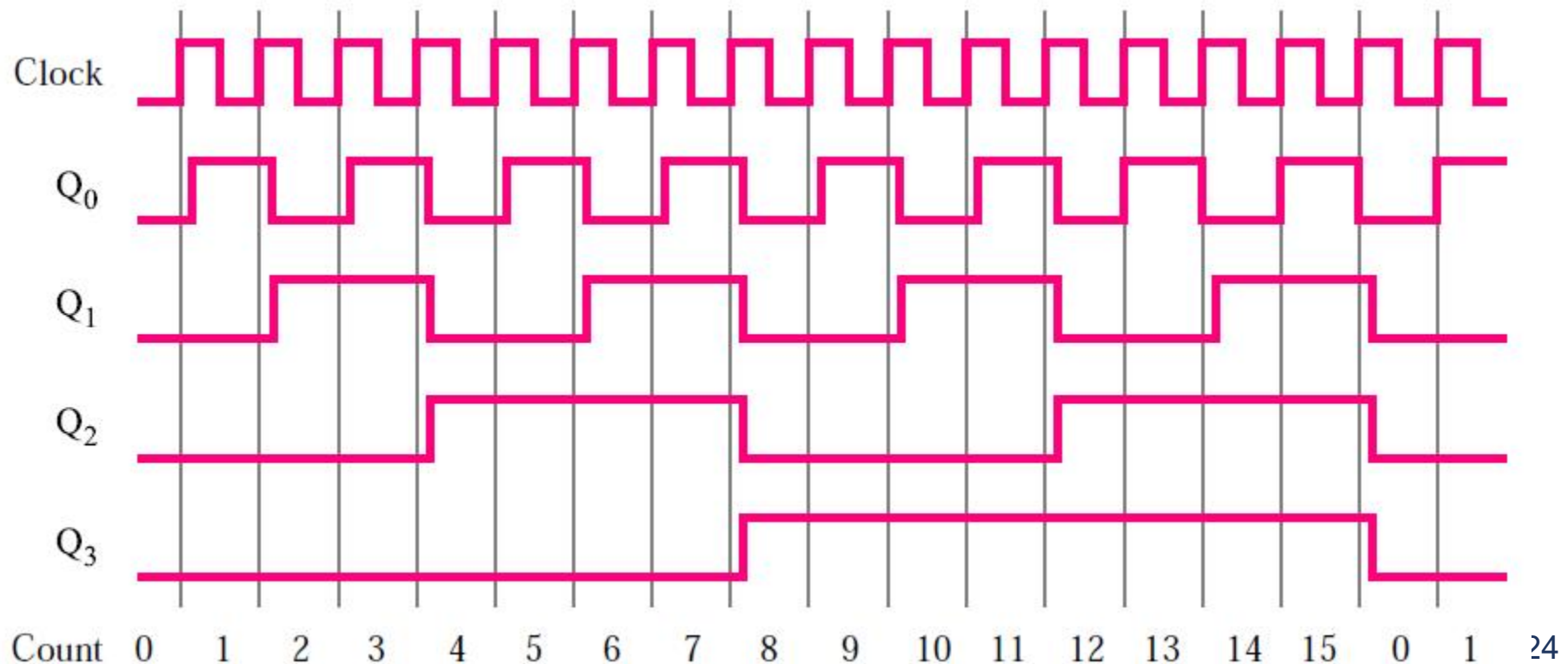
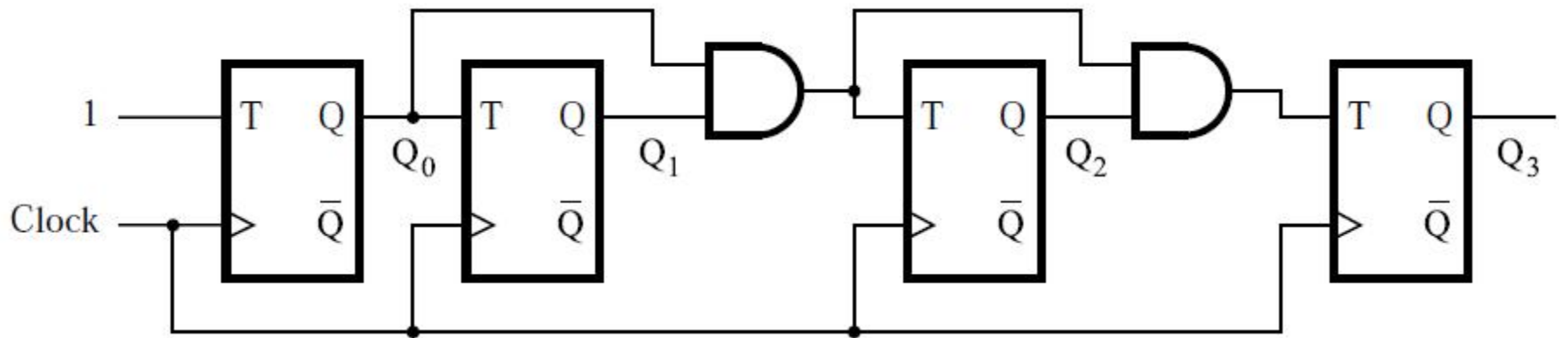
Clock cycle	Q_2	Q_1	Q_0	
0	0	0	0	
1	0	0	1	
2	0	1	0	Q_1 changes
3	0	1	1	
4	1	0	0	Q_2 changes
5	1	0	1	
6	1	1	0	
7	1	1	1	
8	0	0	0	



Can you design a 4 bit counter sync up counter
using either T FF or JK FF?



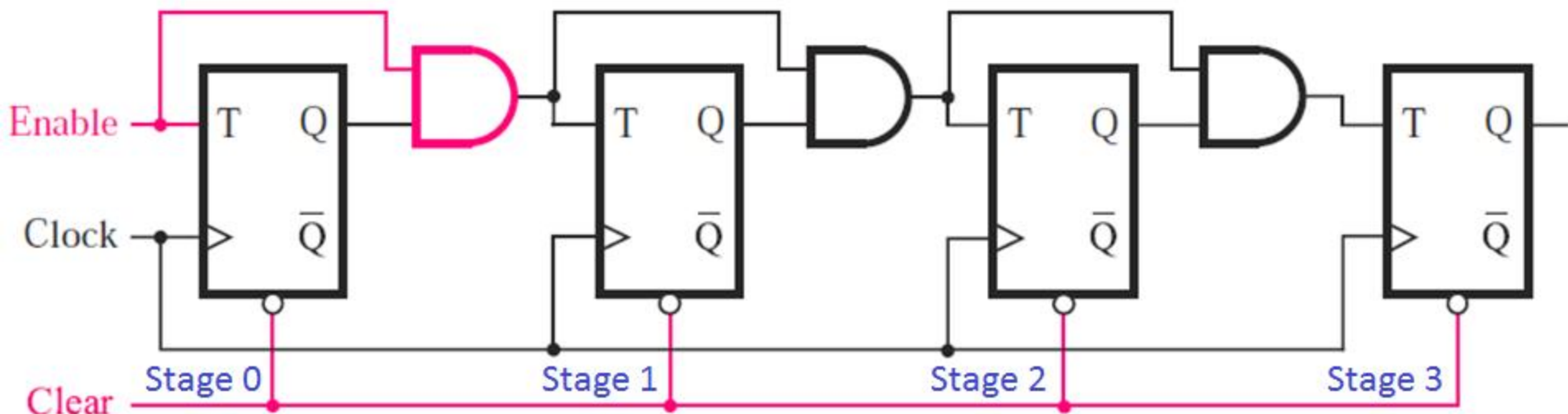
Synchronous 4-bit up-counter with T flip-flops



Enable and clear capabilities

4-bit synchronous up-counter with T flip-flops

- Clear=0: reset counter output to 0 (clear is asynchronous)
- T inputs: $T_0 = \text{Enable}$, $T_1 = Q_0 \cdot \text{Enable}$, $T_2 = Q_1 \cdot Q_0 \cdot \text{Enable}$, $T_3 = Q_2 \cdot Q_1 \cdot Q_0 \cdot \text{Enable}$
- Enable=1: normal operation as counter
- Enable=0: hold current state (all T inputs are 0)



A 4-Bit Synchronous Decade Counter

Synchronous reset technique (avoid glitches):

TABLE 9-5

States of a BCD decade counter.

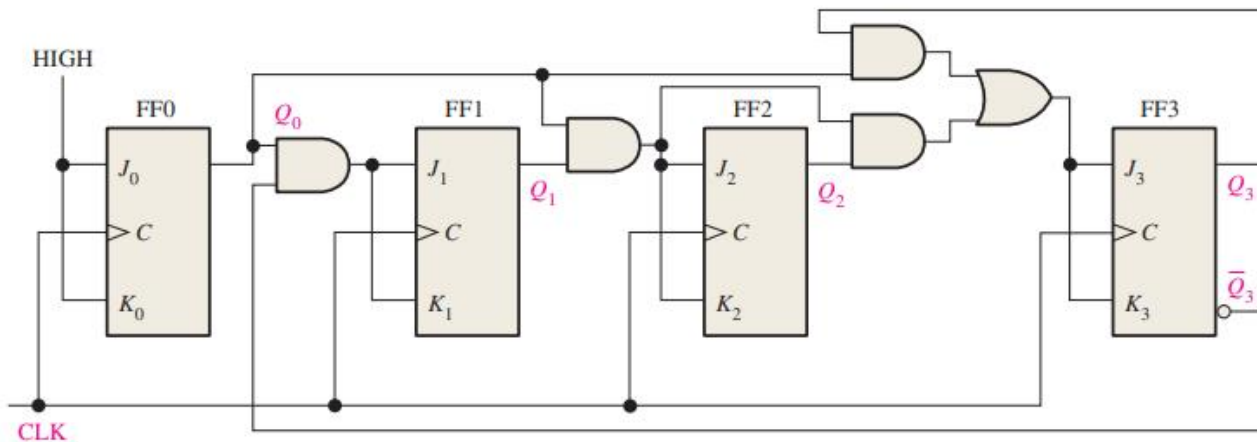
Clock Pulse	Q_3	Q_2	Q_1	Q_0
Initially	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10 (recycles)	0	0	0	0

Exercise: derive J1 J2 J3 using K-Map

$$J_1 = K_1 = Q_0 \bar{Q}_3 \quad \text{Flip at 1 3 5 7 (but not 9)}$$

$$J_2 = K_2 = Q_0 Q_1 \quad \text{Flip at 3 7}$$

$$J_3 = K_3 = Q_0 Q_1 Q_2 + Q_0 Q_3 \quad \text{Flip at 7 9}$$



Modularized Design

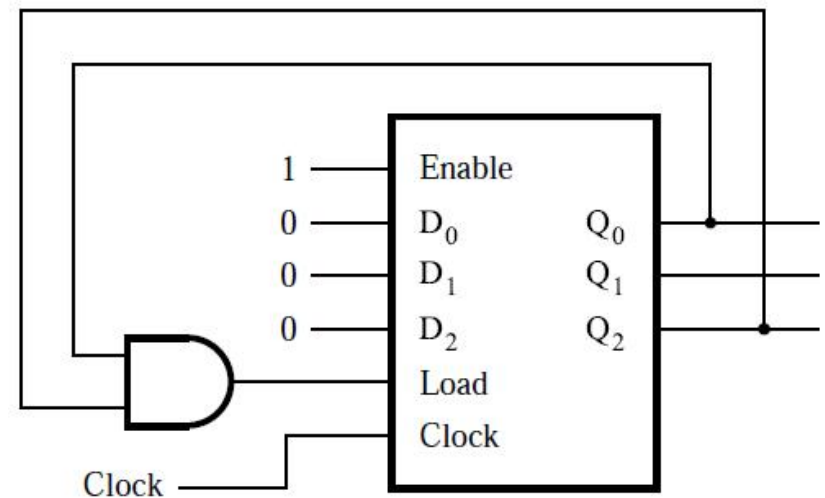
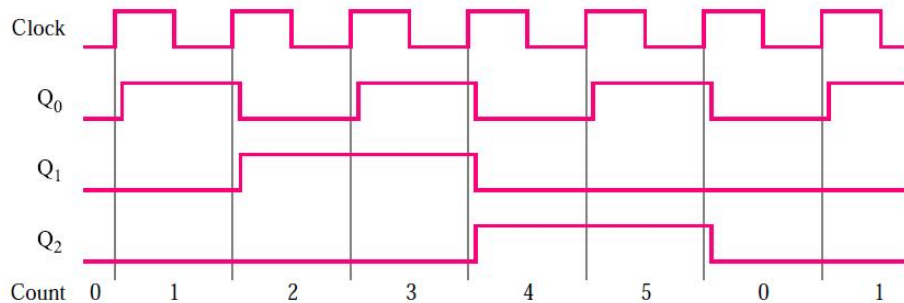
Using counter module to design, simple to implement.

Load : implemented using parallel loading mode

Load condition: $Q_2Q_1Q_0=101$ (count 5, modulus=6)

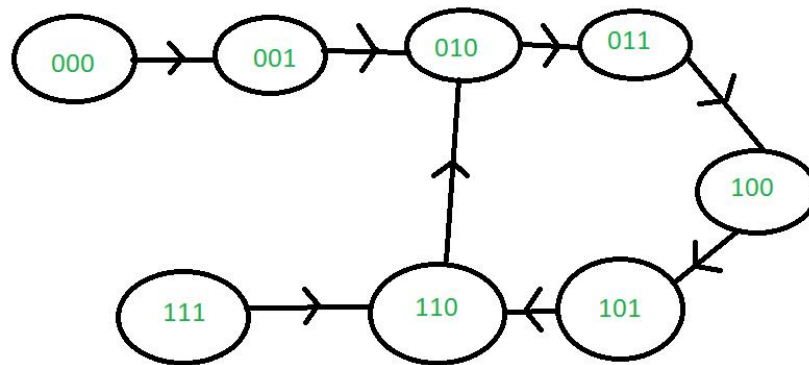
Load=1 when $Q_2=Q_0=1$: load $D_2D_1D_0=000$ (reset counter)

Load is synchronized by clock.

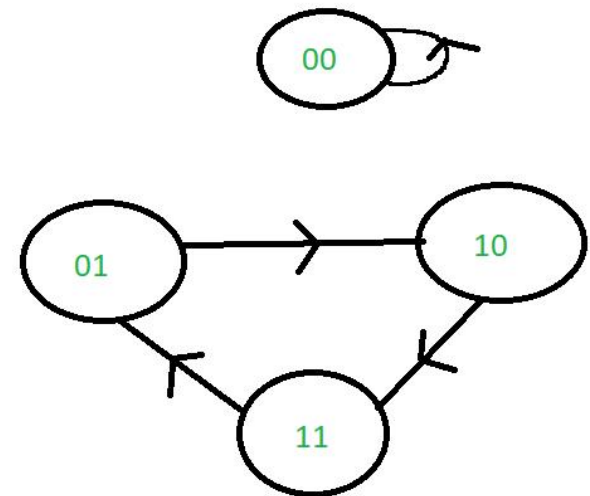


Self-starting counter

Counter is said to be self-starting if it is possible to enter a counter loop irrespective of the initial state.



Capable of self-starting

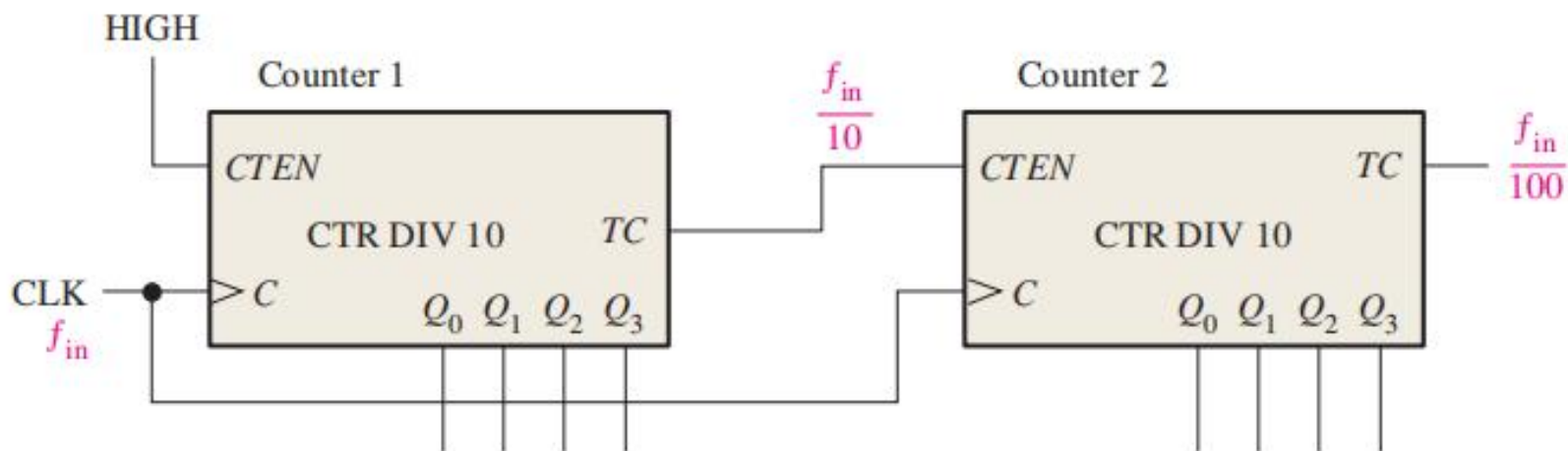


Cannot self-starting

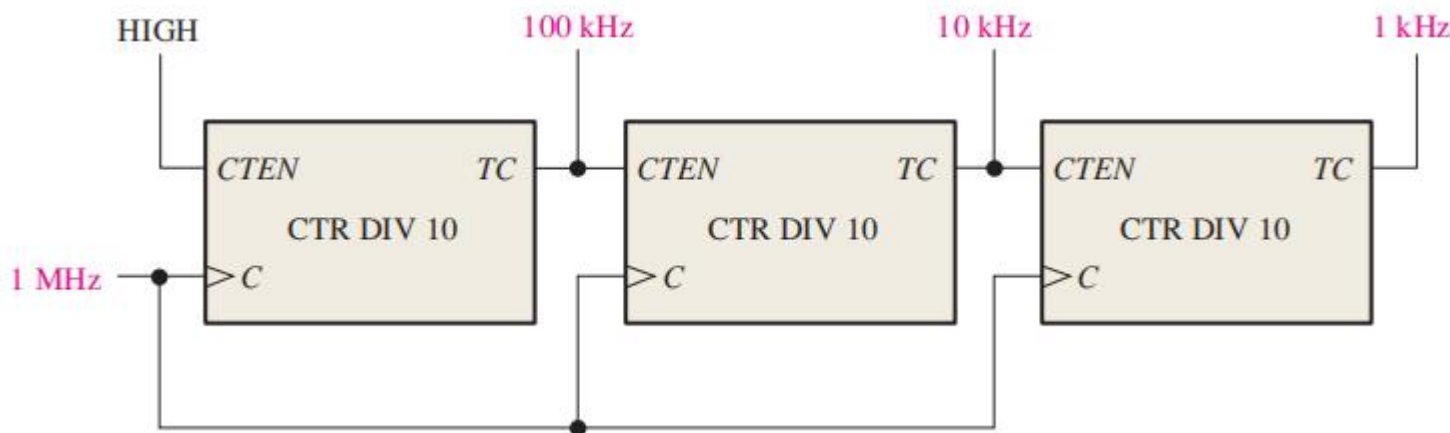


Synchronous Cascading

Cascaded counter



Frequency divider



Code for 3-bit **down-counter**

Modulus: 8

E: enable

L: load

Initial count: modulus-1

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY downcnt IS
    GENERIC ( modulus : INTEGER := 8 ) ;
    PORT ( Clock, L, E : IN    STD_LOGIC ;
          Q           : OUT INTEGER RANGE 0 TO modulus-1 ) ;
END downcnt ;

ARCHITECTURE Behavior OF downcnt IS
    SIGNAL Count : INTEGER RANGE 0 TO modulus-1 ;
BEGIN
    PROCESS
    BEGIN
        WAIT UNTIL (Clock'EVENT AND Clock = '1') ;
        IF L = '1' THEN
            Count <= modulus-1 ;
        ELSE
            IF E = '1' THEN
                Count <= Count-1 ;
            END IF ;
        END IF ;
    END PROCESS;
    Q <= Count ;
END Behavior ;
```



Code for 4-bit synchronous up-counter with asynchronous reset (clear) and enable

Ports

- Resetn: clear
- E: Enable

Signal

- Count: outputs

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
USE ieee.std_logic_unsigned.all ;

ENTITY upcount IS
    PORT ( Clock, Resetn, E : IN  STD_LOGIC ;
          Q : OUT STD_LOGIC_VECTOR (3 DOWNT0 0)) ;
END upcount ;

ARCHITECTURE Behavior OF upcount IS
    SIGNAL Count : STD_LOGIC_VECTOR (3 DOWNT0 0) ;
BEGIN
    PROCESS ( Clock, Resetn )
    BEGIN
        IF Resetn = '0' THEN
            Count <= "0000" ;
        ELSIF (Clock'EVENT AND Clock = '1') THEN
            IF E = '1' THEN
                Count <= Count + 1 ;
            ELSE
                Count <= Count ;
            END IF ;
        END IF ;
    END PROCESS ;
    Q <= Count ;
END Behavior ;
```



Code for 4-bit synchronous up-counter with asynchronous reset and **parallel loading**

L: load

R: parallel data

Range: 4-bit

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY upcount IS
    PORT ( R          : IN      INTEGER RANGE 0 TO 15 ;
          Clock, Resetn, L : IN      STD_LOGIC ;
          Q          : BUFFER INTEGER RANGE 0 TO 15 ) ;
END upcount ;

ARCHITECTURE Behavior OF upcount IS
BEGIN
    PROCESS ( Clock, Resetn )
    BEGIN
        IF Resetn = '0' THEN
            Q <= 0 ;
        ELSIF (Clock'EVENT AND Clock = '1') THEN
            IF L = '1' THEN
                Q <= R ;
            ELSE
                Q <= Q + 1 ;
            END IF;
        END IF;
    END PROCESS;
END Behavior;
```

